

PYTHON-01

Python Programming & Engineering Reference

PART 4 - OBJECT-ORIENTED PROGRAMMING & PYTHON DATA MODEL

Chapters 100-149 • English / Français

Engineering focus / Orientation ingénierie

AHDS | Anissoff Hardware Development System



© AHDS. Portfolio / educational material. Redistribution or reproduction without permission is not authorized.

Contents / Table des matières

Section A — Classes & Objects

100. Object-Oriented Programming Fundamentals
101. Defining Classes
102. Creating Objects & Instances
103. Instance Initialization — __init__()
104. Instance Attributes
105. Class Attributes
106. Instance Methods & self
107. Class Methods & @classmethod
108. Static Methods & @staticmethod
109. Attribute Access & Object State
110. Public, Protected & Private Naming Conventions
111. Properties — @property
112. getattr(), setattr(), hasattr() & delattr()
113. Dynamic Attributes

Section B — Inheritance & Polymorphism

114. Inheritance Fundamentals
115. Single Inheritance
116. Multiple Inheritance
117. Method Resolution Order — MRO
118. super()
119. Method Overriding
120. Polymorphism & Duck Typing
121. Composition vs Inheritance
122. Abstract Base Classes — ABC
123. Protocols & Structural Typing
124. Interface-Oriented Engineering Design

Section C — Dataclasses & Engineering Models

125. Dataclasses Fundamentals
126. Dataclass Fields, Defaults & Factories
127. Immutable/Frozen Dataclasses
128. Ordering, Equality & Hashing in Dataclasses
129. Structured Engineering Data Models

Section D — Object Semantics

130. Identity, Equality & Object Semantics
131. Hashing Custom Objects
132. Copying Object Graphs & Custom Objects
133. Object Lifetime & Garbage Collection Concepts

Section E — Python Data Model & Special Methods

134. Special Methods / Dunder Methods
135. Object String Representation
136. Comparison Special Methods
137. Operator Overloading
138. Container Protocols
139. Iterator Protocol
140. Callable Objects — __call__()
141. Context Manager Protocol
142. Customizing Attribute Access



© AHDS. Portfolio / educational material. Redistribution or reproduction without permission is not authorized.



143. Descriptors
144. __slots__
145. Object Creation — __new__()
146. Metaclasses
147. Class Decorators & Dynamic Class Construction

Section F — Engineering OOP

148. OOP for Hardware, Drivers & Instruments
149. Complete Object-Oriented Engineering Architecture



© AHDS. Portfolio / educational material. Redistribution or reproduction without permission is not authorized.



Chapter 100 - Object-Oriented Programming Fundamentals

Section A — Classes & Objects

ENGLISH	FRANÇAIS
What is Object-Oriented Programming Fundamentals? OOP organizes behavior and state around objects with explicit responsibilities.	Quel est le principe présenté dans ce chapitre ? La POO organise le comportement et l'état autour d'objets ayant des responsabilités explicites.
Why does it matter in engineering software? Use classes when state, invariants and operations belong together; do not wrap every function in a class.	**Pourquoi est-ce important dans un logiciel d'ingénierie ?** Utilisez des classes lorsque l'état, les invariants et les opérations vont ensemble; n'encapsulez pas chaque fonction dans une classe.
What design rule should be remembered? Prefer an object only when state and behavior require a shared invariant.	**Quelle règle de conception faut-il retenir ?** N'utiliser un objet que si état et comportement partagent un invariant.

Engineering Notes / Notes d'ingénierie

Engineering check: Prefer an object only when state and behavior require a shared invariant.	Vérification d'ingénierie : N'utiliser un objet que si état et comportement partagent un invariant.

Shared engineering example - code remains English only

```
class Sensor:
    def __init__(self, channel):
        self.channel = channel
    def read(self):
        return 2.5  # simulated voltage
print(Sensor(0).read())
```



© AHDS. Portfolio / educational material. Redistribution or reproduction without permission is not authorized.



Chapter 101 - Defining Classes

Section A — Classes & Objects

ENGLISH	FRANÇAIS
What is Defining Classes? A class statement creates a class object and a namespace for attributes and methods.	Quel est le principe présenté dans ce chapitre ? L'instruction class crée un objet classe et un espace de noms pour les attributs et méthodes.
Why does it matter in engineering software? Keep the public surface small and name the class after the engineering concept it represents.	**Pourquoi est-ce important dans un logiciel d'ingénierie ?** Gardez une interface publique réduite et nommez la classe selon le concept d'ingénierie représenté.
What design rule should be remembered? Keep class responsibilities narrow and give instances explicit state.	**Quelle règle de conception faut-il retenir ?** Limiter les responsabilités et rendre l'état des instances explicite.

Engineering Notes / Notes d'ingénierie

Engineering check: Keep class responsibilities narrow and give instances explicit state.	Vérification d'ingénierie : Limiter les responsabilités et rendre l'état des instances explicite.

Shared engineering example - code remains English only

```
class Relay:
    def __init__(self):
        self.state = False
    def on(self):
        self.state = True
a = Relay(); b = Relay(); a.on()
assert a.state and not b.state
```



© AHDS. Portfolio / educational material. Redistribution or reproduction without permission is not authorized.



Chapter 102 - Creating Objects & Instances

Section A — Classes & Objects

ENGLISH	FRANÇAIS
What is Creating Objects & Instances? Calling a class normally creates an instance and runs its initialization path.	Quel est le principe présenté dans ce chapitre ? Appeler une classe crée normalement une instance et exécute son chemin d'initialisation.
Why does it matter in engineering software? Separate object construction from expensive hardware connection when startup must be controlled.	**Pourquoi est-ce important dans un logiciel d'ingénierie ?** Séparez la construction de l'objet de la connexion matérielle coûteuse lorsque le démarrage doit être contrôlé.
What design rule should be remembered? Separate construction, configuration and physical device connection.	**Quelle règle de conception faut-il retenir ?** Séparer construction, configuration et connexion matérielle.

Engineering Notes / Notes d'ingénierie

Engineering check: Separate construction, configuration and physical device connection.	Vérification d'ingénierie : Séparer construction, configuration et connexion matérielle.

Shared engineering example - code remains English only

```
class Meter:
    def __init__(self, port):
        self.port = port
a = Meter("COM4"); b = Meter("COM4")
assert a is not b
```



© AHDS. Portfolio / educational material. Redistribution or reproduction without permission is not authorized.



Chapter 103 - Instance Initialization — __init__()

Section A — Classes & Objects

ENGLISH	FRANÇAIS
What is Instance Initialization — __init__()? __init__ initializes an already-created instance; it does not create the object itself.	Quel est le principe présenté dans ce chapitre ? __init__ initialise une instance déjà créée; il ne crée pas l'objet lui-même.
Why does it matter in engineering software? Validate configuration early, but avoid hidden I/O unless the API explicitly promises it.	**Pourquoi est-ce important dans un logiciel d'ingénierie ?** Validez la configuration tôt, mais évitez les E/S cachées sauf si l'API les annonce explicitement.
What design rule should be remembered? Validate constructor inputs before publishing a usable object.	**Quelle règle de conception faut-il retenir ?** Valider les paramètres avant de rendre l'objet utilisable.

Engineering Notes / Notes d'ingénierie

Engineering check: Validate constructor inputs before publishing a usable object.	Vérification d'ingénierie : Valider les paramètres avant de rendre l'objet utilisable.

Shared engineering example - code remains English only

```
class PSU:
  def __init__(self, voltage=5.0):
    if voltage < 0: raise ValueError('voltage')
    self.voltage = voltage
```



© AHDS. Portfolio / educational material. Redistribution or reproduction without permission is not authorized.



Chapter 104 - Instance Attributes

Section A — Classes & Objects

ENGLISH	FRANÇAIS
What is Instance Attributes? Instance attributes store state on each individual object.	Quel est le principe présenté dans ce chapitre ? Les attributs d'instance stockent l'état de chaque objet individuel.
Why does it matter in engineering software? Use them for per-device address, calibration and runtime state; establish invariants in initialization.	**Pourquoi est-ce important dans un logiciel d'ingénierie ?** Utilisez-les pour adresse, étalonnage et état d'exécution propres à chaque appareil; établissez les invariants à l'initialisation.
What design rule should be remembered? Keep per-device runtime state on instances, never in shared mutable fields.	**Quelle règle de conception faut-il retenir ?** Conserver l'état propre au matériel dans chaque instance.

Engineering Notes / Notes d'ingénierie

Engineering check: Keep per-device runtime state on instances, never in shared mutable fields.	Vérification d'ingénierie : Conserver l'état propre au matériel dans chaque instance.

Shared engineering example - code remains English only

```
class Device:
    def __init__(self, address):
        self.address = address
        self.connected = False
a = Device(1); b = Device(2)
```



© AHDS. Portfolio / educational material. Redistribution or reproduction without permission is not authorized.



Chapter 105 - Class Attributes

Section A — Classes & Objects

ENGLISH	FRANÇAIS
What is Class Attributes? Class attributes live on the class and are shared through attribute lookup unless shadowed by an instance.	Quel est le principe présenté dans ce chapitre ? Les attributs de classe vivent sur la classe et sont partagés via la recherche d'attribut sauf masquage par une instance.
Why does it matter in engineering software? Good uses include immutable constants and metadata; avoid shared mutable runtime state.	**Pourquoi est-ce important dans un logiciel d'ingénierie ?** Les constantes immuables et métadonnées sont de bons usages; évitez l'état d'exécution mutable partagé.
What design rule should be remembered? Use class attributes for shared constants, not mutable device buffers.	**Quelle règle de conception faut-il retenir ?** Réserver les attributs de classe aux constantes, pas aux tampons mutables.

Engineering Notes / Notes d'ingénierie

Engineering check: Use class attributes for shared constants, not mutable device buffers.	Vérification d'ingénierie : Réserver les attributs de classe aux constantes, pas aux tampons mutables.

Shared engineering example - code remains English only

```
class ADC:
    RESOLUTION_BITS = 16
```



© AHDS. Portfolio / educational material. Redistribution or reproduction without permission is not authorized.



Chapter 106 - Instance Methods & self

Section A — Classes & Objects

ENGLISH	FRANÇAIS
What is Instance Methods & self? Instance methods receive the instance conventionally named self and operate on its state.	Quel est le principe présenté dans ce chapitre ? Les méthodes d'instance reçoivent l'instance conventionnellement nommée self et opèrent sur son état.
Why does it matter in engineering software? Keep methods cohesive: a driver method should represent one clear device operation or query.	**Pourquoi est-ce important dans un logiciel d'ingénierie ?** Gardez les méthodes cohésives: une méthode de pilote doit représenter une opération ou requête claire.
What design rule should be remembered? Methods should perform one well-defined operation with explicit failure modes.	**Quelle règle de conception faut-il retenir ?** Une méthode doit réaliser une opération et définir ses échecs.

Engineering Notes / Notes d'ingénierie

Engineering check: Methods should perform one well-defined operation with explicit failure modes.	Vérification d'ingénierie : Une méthode doit réaliser une opération et définir ses échecs.

Shared engineering example - code remains English only

```
class Meter:
    def __init__(self, response):
        self.response = response
    def read_voltage(self):
        return float(self.response)
assert Meter("3.30").read_voltage() == 3.3
```



© AHDS. Portfolio / educational material. Redistribution or reproduction without permission is not authorized.



Chapter 107 - Class Methods & @classmethod

Section A — Classes & Objects

ENGLISH	FRANÇAIS
What is Class Methods & @classmethod? @classmethod receives the class as cls and is useful for alternate constructors and class-wide behavior.	Quel est le principe présenté dans ce chapitre ? @classmethod reçoit la classe via cls et sert aux constructeurs alternatifs et comportements liés à la classe.
Why does it matter in engineering software? Alternate constructors can parse configuration without hard-coding subclass names.	**Pourquoi est-ce important dans un logiciel d'ingénierie ?** Les constructeurs alternatifs peuvent analyser une configuration sans coder en dur les noms de sous-classes.
What design rule should be remembered? Make alternate constructors validate configuration and preserve subclass type.	**Quelle règle de conception faut-il retenir ?** Valider la configuration et préserver le type de sous-classe.

Engineering Notes / Notes d'ingénierie

Engineering check: Make alternate constructors validate configuration and preserve subclass type.	Vérification d'ingénierie : Valider la configuration et préserver le type de sous-classe.

Shared engineering example - code remains English only

```
class Device:
    def __init__(self, address):
        self.address = address
    @classmethod
    def from_config(cls, cfg):
        return cls(cfg["address"])
dev = Device.from_config({"address": 7})
```



© AHDS. Portfolio / educational material. Redistribution or reproduction without permission is not authorized.



Chapter 108 - Static Methods & @staticmethod

Section A — Classes & Objects

ENGLISH	FRANÇAIS
What is Static Methods & @staticmethod? @staticmethod defines a function in the class namespace that needs neither instance nor class state.	Quel est le principe présenté dans ce chapitre ? @staticmethod définit une fonction dans l'espace de noms de la classe qui n'a besoin ni de l'instance ni de l'état de classe.
Why does it matter in engineering software? Use sparingly; a module-level function is often clearer unless the operation belongs conceptually to the class API.	**Pourquoi est-ce important dans un logiciel d'ingénierie ?** Utilisez-la avec parcimonie; une fonction de module est souvent plus claire sauf si l'opération appartient conceptuellement à l'API de classe.
What design rule should be remembered? Choose static methods only when the operation belongs to the class API.	**Quelle règle de conception faut-il retenir ?** N'employer staticmethod que si la fonction appartient à l'API de classe.

Engineering Notes / Notes d'ingénierie

Engineering check: Choose static methods only when the operation belongs to the class API.	Vérification d'ingénierie : N'employer staticmethod que si la fonction appartient à l'API de classe.

Shared engineering example - code remains English only

```
class ADC:
    @staticmethod
    def volts_from_code(code, vref=3.3):
        if not 0 <= code <= 65535: raise ValueError(code)
        return code * vref / 65535
v = ADC.volts_from_code(32768)
```



© AHDS. Portfolio / educational material. Redistribution or reproduction without permission is not authorized.



Chapter 109 - Attribute Access & Object State

Section A — Classes & Objects

ENGLISH	FRANÇAIS
What is Attribute Access & Object State? Attribute lookup follows Python's data model across instance, class and descriptors.	Quel est le principe présenté dans ce chapitre ? La recherche d'attribut suit le modèle de données Python entre instance, classe et descripteurs.
Why does it matter in engineering software? Do not expose half-valid state. Prefer methods/properties when a state transition needs validation.	**Pourquoi est-ce important dans un logiciel d'ingénierie ?** N'exposez pas d'état partiellement valide. Préférez méthodes/propriétés lorsqu'une transition d'état exige validation.
What design rule should be remembered? Avoid exposing partially initialized states or unchecked transitions.	**Quelle règle de conception faut-il retenir ?** Éviter les états incomplets et les transitions non contrôlées.

Engineering Notes / Notes d'ingénierie

Engineering check: Avoid exposing partially initialized states or unchecked transitions.	Vérification d'ingénierie : Éviter les états incomplets et les transitions non contrôlées.

Shared engineering example - code remains English only

```
class Device:
    def __init__(self):
        self.mode = "IDLE"
device = Device(); device.mode = "AUTO"
assert device.mode == "AUTO"
```



© AHDS. Portfolio / educational material. Redistribution or reproduction without permission is not authorized.



Chapter 110 - Public, Protected & Private Naming Conventions

Section A — Classes & Objects

ENGLISH	FRANÇAIS
What is Public, Protected & Private Naming Conventions? Python uses naming conventions rather than enforced private access; double-leading underscores trigger name mangling.	Quel est le principe présenté dans ce chapitre ? Python utilise des conventions de nommage plutôt qu'un accès privé imposé; deux underscores initiaux déclenchent le name mangling.
Why does it matter in engineering software? A single underscore communicates non-public implementation detail; design a stable public driver API instead of relying on secrecy.	**Pourquoi est-ce important dans un logiciel d'ingénierie ?** Un underscore indique un détail non public; concevez une API de pilote stable plutôt que de compter sur le secret.
What design rule should be remembered? Name mangling is collision avoidance, not security or access control.	**Quelle règle de conception faut-il retenir ?** Le masquage des noms n'est ni sécurité ni contrôle d'accès.

Engineering Notes / Notes d'ingénierie

Engineering check: Name mangling is collision avoidance, not security or access control.	Vérification d'ingénierie : Le masquage des noms n'est ni sécurité ni contrôle d'accès.

Shared engineering example - code remains English only

```
class Device:
    def __init__(self, transport, token):
        self._transport = transport  # internal API
        self.__token = token  # name-mangled
device = Device("USB", "demo")
```



© AHDS. Portfolio / educational material. Redistribution or reproduction without permission is not authorized.



Chapter 111 - Properties — @property

Section A — Classes & Objects

ENGLISH	FRANÇAIS
What is Properties — @property? A property exposes managed attribute syntax while keeping validation or computation behind an API.	**Qu'est-ce que Properties — @property ?** Une propriété expose une syntaxe d'attribut géré tout en conservant validation ou calcul derrière une API.
Why does it matter in engineering software? Properties are useful for units and limits, but surprising network or serial I/O in a property is usually poor design.	**Pourquoi est-ce important dans un logiciel d'ingénierie ?** Les propriétés sont utiles pour les unités et limites, mais des E/S réseau ou série surprenantes dans une propriété sont généralement un mauvais choix.
What design rule should be remembered? Properties should stay inexpensive; perform explicit I/O through methods.	**Quelle règle de conception faut-il retenir ?** Garder les propriétés légères; utiliser des méthodes pour les E/S.

Engineering Notes / Notes d'ingénierie

Engineering check: Properties should stay inexpensive; perform explicit I/O through methods.	Vérification d'ingénierie : Garder les propriétés légères; utiliser des méthodes pour les E/S.

Shared engineering example - code remains English only

```
class Supply:
  def __init__(self): self._voltage = 0.0
  @property
  def voltage(self): return self._voltage
  @voltage.setter
  def voltage(self, v):
    if not 0 <= v <= 30: raise ValueError(v)
    self._voltage = float(v)
```



© AHDS. Portfolio / educational material. Redistribution or reproduction without permission is not authorized.



Chapter 112 - getattr(), setattr(), hasattr() & delattr()

Section A — Classes & Objects

ENGLISH	FRANÇAIS
What is getattr(), setattr(), hasattr() & delattr()? getattr, setattr, hasattr and delattr perform attribute operations using names available at runtime.	**Qu'est-ce que getattr(), setattr(), hasattr() & delattr() ?** getattr, setattr, hasattr et delattr effectuent des opérations d'attribut à partir de noms connus à l'exécution.
Why does it matter in engineering software? Useful for adapters and configuration mapping, but validate names before dynamic mutation.	**Pourquoi est-ce important dans un logiciel d'ingénierie ?** Utile pour adaptateurs et mappage de configuration, mais validez les noms avant mutation dynamique.
What design rule should be remembered? Do not call unknown dynamic attributes without checking the contract.	**Quelle règle de conception faut-il retenir ?** Ne pas appeler des attributs dynamiques sans vérifier le contrat.

Engineering Notes / Notes d'ingénierie

Engineering check: Do not call unknown dynamic attributes without checking the contract.	Vérification d'ingénierie : Ne pas appeler des attributs dynamiques sans vérifier le contrat.

Shared engineering example - code remains English only

```
class Device:
    def reset(self): return "ready"
device = Device()
method = getattr(device, "reset", None)
if callable(method): method()
```



© AHDS. Portfolio / educational material. Redistribution or reproduction without permission is not authorized.



Chapter 113 - Dynamic Attributes

Section A — Classes & Objects

ENGLISH	FRANÇAIS
What is Dynamic Attributes? Python objects can support attributes chosen at runtime when their layout allows it.	Quel est le principe présenté dans ce chapitre ? Les objets Python peuvent prendre en charge des attributs choisis à l'exécution lorsque leur disposition le permet.
Why does it matter in engineering software? Dynamic metadata is convenient, but explicit schemas are safer for production configuration and traceability.	**Pourquoi est-ce important dans un logiciel d'ingénierie ?** Les métadonnées dynamiques sont pratiques, mais des schémas explicites sont plus sûrs pour configuration et traçabilité.
What design rule should be remembered? Prefer validated schemas to uncontrolled runtime metadata.	**Quelle règle de conception faut-il retenir ?** Préférer des schémas validés aux métadonnées arbitraires.

Engineering Notes / Notes d'ingénierie

Engineering check: Prefer validated schemas to uncontrolled runtime metadata.	Vérification d'ingénierie : Préférer des schémas validés aux métadonnées arbitraires.

Shared engineering example - code remains English only

```
class Result: pass
result = Result()
setattr(result, "fixture_id", "FX-07")
assert result.fixture_id == "FX-07"
```



© AHDS. Portfolio / educational material. Redistribution or reproduction without permission is not authorized.



Chapter 114 - Inheritance Fundamentals

Section B — Inheritance & Polymorphism

ENGLISH	FRANÇAIS
What is Inheritance Fundamentals? Inheritance models an is-a relationship and reuses or specializes behavior from a base class.	Quel est le principe présenté dans ce chapitre ? L'héritage modélise une relation « est-un » et réutilise ou spécialise le comportement d'une classe de base.
Why does it matter in engineering software? Prefer shallow hierarchies; hardware families often benefit from a stable base interface plus composition.	**Pourquoi est-ce important dans un logiciel d'ingénierie ?** Préférez des hiérarchies peu profondes; les familles matérielles bénéficient souvent d'une interface de base stable et de la composition.
What design rule should be remembered? A subtype must preserve the expectations of its base class.	**Quelle règle de conception faut-il retenir ?** Un sous-type doit respecter le contrat de sa classe de base.

Engineering Notes / Notes d'ingénierie

Engineering check: A subtype must preserve the expectations of its base class.	Vérification d'ingénierie : Un sous-type doit respecter le contrat de sa classe de base.

Shared engineering example - code remains English only

```
class Instrument: pass
class DMM(Instrument): pass
```



© AHDS. Portfolio / educational material. Redistribution or reproduction without permission is not authorized.



Chapter 115 - Single Inheritance

Section B — Inheritance & Polymorphism

ENGLISH	FRANÇAIS
What is Single Inheritance? Single inheritance specializes one base class and is the simplest inheritance form.	Quel est le principe présenté dans ce chapitre ? L'héritage simple spécialise une classe de base et constitue la forme d'héritage la plus simple.
Why does it matter in engineering software? Use it when the subtype truly preserves the base contract.	**Pourquoi est-ce important dans un logiciel d'ingénierie ?** Utilisez-le lorsque le sous-type respecte réellement le contrat de base.
What design rule should be remembered? Use inheritance only where substitution remains behaviorally valid.	**Quelle règle de conception faut-il retenir ?** N'utiliser l'héritage que si la substitution demeure valide.

Engineering Notes / Notes d'ingénierie

Engineering check: Use inheritance only where substitution remains behaviorally valid.	Vérification d'ingénierie : N'utiliser l'héritage que si la substitution demeure valide.

Shared engineering example - code remains English only

```
class DMM:
    def read(self): return 5.0
class SerialDMM(DMM):
    baudrate = 115200
assert SerialDMM().read() == 5.0
```



© AHDS. Portfolio / educational material. Redistribution or reproduction without permission is not authorized.



Chapter 116 - Multiple Inheritance

Section B — Inheritance & Polymorphism

ENGLISH	FRANÇAIS
What is Multiple Inheritance? Multiple inheritance combines behavior from more than one base and therefore depends on cooperative MRO behavior.	Quel est le principe présenté dans ce chapitre ? L'héritage multiple combine le comportement de plusieurs bases et dépend donc d'un MRO coopératif.
Why does it matter in engineering software? Use it carefully for mixins; avoid using it to merge unrelated stateful device classes.	**Pourquoi est-ce important dans un logiciel d'ingénierie ?** Utilisez-le avec prudence pour les mixins; évitez de fusionner des classes d'appareils avec états sans lien.
What design rule should be remembered? Prefer cooperative mixins over stateful multiple-inheritance diamonds.	**Quelle règle de conception faut-il retenir ?** Préférer les mixins coopératifs aux hiérarchies ambiguës.

Engineering Notes / Notes d'ingénierie

Engineering check: Prefer cooperative mixins over stateful multiple-inheritance diamonds.	Vérification d'ingénierie : Préférer les mixins coopératifs aux hiérarchies ambiguës.

Shared engineering example - code remains English only

```
class Instrument:
    def read(self): return 5.0
class LoggingMixin:
    def log(self, message): return str(message)
class VisaInstrument(LoggingMixin, Instrument): pass
```



© AHDS. Portfolio / educational material. Redistribution or reproduction without permission is not authorized.



Chapter 117 - Method Resolution Order — MRO

Section B — Inheritance & Polymorphism

ENGLISH	FRANÇAIS
What is Method Resolution Order — MRO? MRO defines the deterministic order Python uses to resolve attributes in an inheritance hierarchy.	Quel est le principe présenté dans ce chapitre ? Le MRO définit l'ordre déterministe utilisé par Python pour résoudre les attributs dans une hiérarchie d'héritage.
Why does it matter in engineering software? Inspect Class.__mro__ when cooperative multiple inheritance behaves unexpectedly.	**Pourquoi est-ce important dans un logiciel d'ingénierie ?** Inspectez Class.__mro__ lorsque l'héritage multiple coopératif se comporte de façon inattendue.
What design rule should be remembered? Inspect the C3 linearization before depending on super() dispatch.	**Quelle règle de conception faut-il retenir ?** Vérifier la linéarisation C3 avant de dépendre de super().

Engineering Notes / Notes d'ingénierie

Engineering check: Inspect the C3 linearization before depending on super() dispatch.	Vérification d'ingénierie : Vérifier la linéarisation C3 avant de dépendre de super().

Shared engineering example - code remains English only

```
class A: pass
class B(A): pass
class C(A): pass
class D(B, C): pass
assert D.__mro__ == (D, B, C, A, object)
```



© AHDS. Portfolio / educational material. Redistribution or reproduction without permission is not authorized.



Chapter 118 - super()

Section B — Inheritance & Polymorphism

ENGLISH	FRANÇAIS
What is super()? super() delegates to the next implementation in the MRO rather than simply naming a parent class.	**Qu'est-ce que super() ?** super() délègue à l'implémentation suivante dans le MRO au lieu de simplement nommer une classe parente.
Why does it matter in engineering software? Use compatible signatures in cooperative hierarchies.	**Pourquoi est-ce important dans un logiciel d'ingénierie ?** Utilisez des signatures compatibles dans les hiérarchies coopératives.
What design rule should be remembered? Use compatible signatures and call super() cooperatively.	**Quelle règle de conception faut-il retenir ?** Employer des signatures compatibles et des appels super() coopératifs.

Engineering Notes / Notes d'ingénierie

Engineering check: Use compatible signatures and call super() cooperatively.	Vérification d'ingénierie : Employer des signatures compatibles et des appels super() coopératifs.

Shared engineering example - code remains English only

```
class Instrument:
    def __init__(self): self.ready = True
class DMM(Instrument):
    def __init__(self, address):
        super().__init__()
        self.address = address
```



© AHDS. Portfolio / educational material. Redistribution or reproduction without permission is not authorized.



Chapter 119 - Method Overriding

Section B — Inheritance & Polymorphism

ENGLISH	FRANÇAIS
What is Method Overriding? Overriding replaces or specializes inherited behavior in a subclass.	Quel est le principe présenté dans ce chapitre ? La redéfinition remplace ou spécialise un comportement hérité dans une sous-classe.
Why does it matter in engineering software? Preserve the documented contract, including return types and failure expectations.	**Pourquoi est-ce important dans un logiciel d'ingénierie ?** Préservez le contrat documenté, y compris types de retour et attentes en cas d'échec.
What design rule should be remembered? Overrides must preserve units, exception contracts and result meaning.	**Quelle règle de conception faut-il retenir ?** Conserver unités, exceptions et sens des résultats redéfinis.

Engineering Notes / Notes d'ingénierie

Engineering check: Overrides must preserve units, exception contracts and result meaning.	Vérification d'ingénierie : Conserver unités, exceptions et sens des résultats redéfinis.

Shared engineering example - code remains English only

```
class DMM:
    def read(self): return 0.0
class SimDMM(DMM):
    def read(self): return 5.001
assert SimDMM().read() == 5.001
```



© AHDS. Portfolio / educational material. Redistribution or reproduction without permission is not authorized.



Chapter 120 - Polymorphism & Duck Typing

Section B — Inheritance & Polymorphism

ENGLISH	FRANÇAIS
What is Polymorphism & Duck Typing? Duck typing focuses on supported behavior rather than nominal class ancestry.	Quel est le principe présenté dans ce chapitre ? Le duck typing se concentre sur le comportement pris en charge plutôt que sur l'ascendance nominale.
Why does it matter in engineering software? This is powerful for test doubles: a simulator can satisfy the same operational contract as real hardware.	**Pourquoi est-ce important dans un logiciel d'ingénierie ?** C'est puissant pour les doublures de test: un simulateur peut satisfaire le même contrat opérationnel que le matériel réel.
What design rule should be remembered? Test behavior at the interface, not the ancestry of the test double.	**Quelle règle de conception faut-il retenir ?** Tester le comportement de l'interface, non l'héritage du simulateur.

Engineering Notes / Notes d'ingénierie

Engineering check: Test behavior at the interface, not the ancestry of the test double.	Vérification d'ingénierie : Tester le comportement de l'interface, non l'héritage du simulateur.

Shared engineering example - code remains English only

```
class SimulatedMeter:
    def read(self): return 5.001
def acquire(device):
    return device.read()
assert acquire(SimulatedMeter()) == 5.001
```



© AHDS. Portfolio / educational material. Redistribution or reproduction without permission is not authorized.



Chapter 121 - Composition vs Inheritance

Section B — Inheritance & Polymorphism

ENGLISH	FRANÇAIS
What is Composition vs Inheritance? Composition builds objects from collaborating components and usually reduces inheritance coupling.	**Qu’est-ce que Composition vs Inheritance ?** La composition construit des objets à partir de composants collaborateurs et réduit généralement le couplage d'héritage.
Why does it matter in engineering software? For engineering systems, compose transport, protocol, calibration and device behavior separately.	**Pourquoi est-ce important dans un logiciel d’ingénierie ?** Pour les systèmes d'ingénierie, composez séparément transport, protocole, étalonnage et comportement de l'appareil.
What design rule should be remembered? Inject transport and protocol objects rather than subclassing each combination.	**Quelle règle de conception faut-il retenir ?** Injecter transport et protocole plutôt que multiplier les sous-classes.

Engineering Notes / Notes d’ingénierie

Engineering check: Inject transport and protocol objects rather than subclassing each combination.	Vérification d’ingénierie : Injecter transport et protocole plutôt que multiplier les sous-classes.

Shared engineering example - code remains English only

```
class Driver:
  def __init__(self, transport): self.transport = transport
```



© AHDS. Portfolio / educational material. Redistribution or reproduction without permission is not authorized.



Chapter 122 - Abstract Base Classes — ABC

Section B — Inheritance & Polymorphism

ENGLISH	FRANÇAIS
What is Abstract Base Classes — ABC? ABC can define required abstract operations and shared concrete behavior.	Quel est le principe présenté dans ce chapitre ? ABC peut définir des opérations abstraites obligatoires et un comportement concret partagé.
Why does it matter in engineering software? Use an ABC when runtime-enforced nominal structure is useful; otherwise a Protocol may be lighter.	**Pourquoi est-ce important dans un logiciel d'ingénierie ?** Utilisez une ABC lorsqu'une structure nominale vérifiée à l'exécution est utile; sinon un Protocol peut être plus léger.
What design rule should be remembered? Use ABC for nominal contracts; document failures and lifecycle semantics.	**Quelle règle de conception faut-il retenir ?** Employer ABC pour les contrats nominaux et documenter le cycle de vie.

Engineering Notes / Notes d'ingénierie

Engineering check: Use ABC for nominal contracts; document failures and lifecycle semantics.	Vérification d'ingénierie : Employer ABC pour les contrats nominaux et documenter le cycle de vie.

Shared engineering example - code remains English only

```
from abc import ABC, abstractmethod
class Instrument(ABC):
    @abstractmethod
    def read(self): ...
```



© AHDS. Portfolio / educational material. Redistribution or reproduction without permission is not authorized.



Chapter 123 - Protocols & Structural Typing

Section B — Inheritance & Polymorphism

ENGLISH	FRANÇAIS
What is Protocols & Structural Typing? Protocol expresses structural interfaces for static type checking without requiring inheritance.	Quel est le principe présenté dans ce chapitre ? Protocol exprime des interfaces structurelles pour le typage statique sans imposer l'héritage.
Why does it matter in engineering software? Protocols fit drivers, transports and simulators that share behavior but not implementation ancestry.	**Pourquoi est-ce important dans un logiciel d'ingénierie ?** Les Protocols conviennent aux pilotes, transports et simulateurs partageant un comportement sans ascendance d'implémentation.
What design rule should be remembered? Protocols express structural requirements; they do not validate at runtime.	**Quelle règle de conception faut-il retenir ?** Les Protocols décrivent les exigences sans les valider à l'exécution.

Engineering Notes / Notes d'ingénierie

Engineering check: Protocols express structural requirements; they do not validate at runtime.	Vérification d'ingénierie : Les Protocols décrivent les exigences sans les valider à l'exécution.

Shared engineering example - code remains English only

```
from typing import Protocol
class Readable(Protocol):
    def read(self) -> float: ...
```



© AHDS. Portfolio / educational material. Redistribution or reproduction without permission is not authorized.



Chapter 124 - Interface-Oriented Engineering Design

Section B — Inheritance & Polymorphism

ENGLISH	FRANÇAIS
What is Interface-Oriented Engineering Design? Interface-oriented design makes dependencies target stable capabilities rather than concrete implementations.	Quel est le principe présenté dans ce chapitre ? La conception orientée interface fait dépendre le système de capacités stables plutôt que d'implémentations concrètes.
Why does it matter in engineering software? It enables hardware substitution, simulation and independent testing of application logic.	**Pourquoi est-ce important dans un logiciel d'ingénierie ?** Elle permet substitution matérielle, simulation et test indépendant de la logique applicative.
What design rule should be remembered? Specify the observable hardware contract independently from its driver.	**Quelle règle de conception faut-il retenir ?** Définir le contrat matériel observable indépendamment du pilote.

Engineering Notes / Notes d'ingénierie

Engineering check: Specify the observable hardware contract independently from its driver.	Vérification d'ingénierie : Définir le contrat matériel observable indépendamment du pilote.

Shared engineering example - code remains English only

```
from typing import Protocol
class SupplyInterface(Protocol):
    def set_voltage(self, volts: float) -> None: ...
def verify_supply(psu: SupplyInterface):
    psu.set_voltage(5.0)
```



© AHDS. Portfolio / educational material. Redistribution or reproduction without permission is not authorized.



Chapter 125 - Dataclasses Fundamentals

Section C — Dataclasses & Engineering Models

ENGLISH	FRANÇAIS
What is Dataclasses Fundamentals? @dataclass generates common data-model methods from annotated fields.	Quel est le principe présenté dans ce chapitre ? @dataclass génère des méthodes courantes du modèle de données à partir des champs annotés.
Why does it matter in engineering software? Use dataclasses for measurements, configuration and immutable result records; keep active device behavior in normal classes.	**Pourquoi est-ce important dans un logiciel d'ingénierie ?** Utilisez les dataclasses pour mesures, configuration et résultats; gardez le comportement actif des appareils dans des classes normales.
What design rule should be remembered? Use dataclasses for passive records and define validation where needed.	**Quelle règle de conception faut-il retenir ?** Utiliser les dataclasses pour les données et ajouter la validation nécessaire.

Engineering Notes / Notes d'ingénierie

Engineering check: Use dataclasses for passive records and define validation where needed.	Vérification d'ingénierie : Utiliser les dataclasses pour les données et ajouter la validation nécessaire.

Shared engineering example - code remains English only

```
from dataclasses import dataclass
@dataclass
class Reading:
    volts: float
    valid: bool = True
```



© AHDS. Portfolio / educational material. Redistribution or reproduction without permission is not authorized.



Chapter 126 - Dataclass Fields, Defaults & Factories

Section C — Dataclasses & Engineering Models

ENGLISH	FRANÇAIS
What is Dataclass Fields, Defaults & Factories? field() controls defaults, factories, representation, comparison and initialization behavior.	Quel est le principe présenté dans ce chapitre ? field() contrôle valeurs par défaut, factories, représentation, comparaison et initialisation.
Why does it matter in engineering software? Use default_factory for mutable values so instances do not accidentally share one container.	**Pourquoi est-ce important dans un logiciel d'ingénierie ?** Utilisez default_factory pour les valeurs mutables afin que les instances ne partagent pas accidentellement un conteneur.
What design rule should be remembered? Use default_factory to avoid sharing mutable containers accidentally.	**Quelle règle de conception faut-il retenir ?** Employer default_factory pour éviter les conteneurs mutables partagés.

Engineering Notes / Notes d'ingénierie

Engineering check: Use default_factory to avoid sharing mutable containers accidentally.	Vérification d'ingénierie : Employer default_factory pour éviter les conteneurs mutables partagés.

Shared engineering example - code remains English only

```
from dataclasses import dataclass, field
@dataclass
class Log:
    samples: list[float] = field(default_factory=list)
```



© AHDS. Portfolio / educational material. Redistribution or reproduction without permission is not authorized.



Chapter 127 - Immutable/Frozen Dataclasses

Section C — Dataclasses & Engineering Models

ENGLISH	FRANÇAIS
What is Immutable/Frozen Dataclasses? frozen=True blocks ordinary field assignment and supports value-object style design.	Quel est le principe présenté dans ce chapitre ? frozen=True bloque l'affectation ordinaire des champs et favorise une conception de type objet-valeur.
Why does it matter in engineering software? Frozen configuration objects reduce accidental runtime mutation, though contained mutable objects still need care.	**Pourquoi est-ce important dans un logiciel d'ingénierie ?** Les configurations gelées réduisent les mutations accidentelles, mais les objets mutables contenus exigent encore de l'attention.
What design rule should be remembered? Frozen prevents ordinary reassignment, not mutation of referenced objects.	**Quelle règle de conception faut-il retenir ?** Frozen empêche les réaffectations, pas les mutations internes.

Engineering Notes / Notes d'ingénierie

Engineering check: Frozen prevents ordinary reassignment, not mutation of referenced objects.	Vérification d'ingénierie : Frozen empêche les réaffectations, pas les mutations internes.

Shared engineering example - code remains English only

```
from dataclasses import dataclass
@dataclass(frozen=True)
class Limits:
    vmin: float
    vmax: float
limits = Limits(0.0, 5.0)
```



© AHDS. Portfolio / educational material. Redistribution or reproduction without permission is not authorized.



Chapter 128 - Ordering, Equality & Hashing in Dataclasses

Section C — Dataclasses & Engineering Models

ENGLISH	FRANÇAIS
What is Ordering, Equality & Hashing in Dataclasses? Dataclass equality, ordering and hashing options define value semantics generated from selected fields.	**Qu'est-ce que Ordering, Equality & Hashing in Dataclasses ?** Les options d'égalité, d'ordre et de hachage des dataclasses définissent une sémantique de valeur générée à partir des champs sélectionnés.
Why does it matter in engineering software? Choose semantics deliberately; measurement records with timestamps may compare differently from configuration keys.	**Pourquoi est-ce important dans un logiciel d'ingénierie ?** Choisissez la sémantique délibérément; des mesures horodatées peuvent se comparer différemment de clés de configuration.
What design rule should be remembered? Choose comparison and hash fields to match engineering identity.	**Quelle règle de conception faut-il retenir ?** Choisir les champs de comparaison selon l'identité métier.

Engineering Notes / Notes d'ingénierie

Engineering check: Choose comparison and hash fields to match engineering identity.	Vérification d'ingénierie : Choisir les champs de comparaison selon l'identité métier.

Shared engineering example - code remains English only

```
from dataclasses import dataclass
@dataclass(order=True, frozen=True)
class Point:
    t: float
    value: float
assert Point(1.0, 2.0) < Point(2.0, 1.0)
```



© AHDS. Portfolio / educational material. Redistribution or reproduction without permission is not authorized.



Chapter 129 - Structured Engineering Data Models

Section C — Dataclasses & Engineering Models

ENGLISH	FRANÇAIS
What is Structured Engineering Data Models? Structured engineering models encode units, limits, identity and validity explicitly instead of passing anonymous dictionaries.	Quel est le principe présenté dans ce chapitre ? Les modèles d'ingénierie structurés encodent explicitement unités, limites, identité et validité au lieu de dictionnaires anonymes.
Why does it matter in engineering software? Typed records improve traceability across acquisition, validation, logging and reporting.	**Pourquoi est-ce important dans un logiciel d'ingénierie ?** Les enregistrements typés améliorent la traçabilité entre acquisition, validation, journalisation et rapport.
What design rule should be remembered? Represent measurement units and validity explicitly in typed records.	**Quelle règle de conception faut-il retenir ?** Expliciter les unités et la validité dans les enregistrements typés.

Engineering Notes / Notes d'ingénierie

Engineering check: Represent measurement units and validity explicitly in typed records.	Vérification d'ingénierie : Expliciter les unités et la validité dans les enregistrements typés.

Shared engineering example - code remains English only

```
from dataclasses import dataclass
@dataclass(frozen=True)
class Measurement:
    value: float
    unit: str
    channel: int
```



© AHDS. Portfolio / educational material. Redistribution or reproduction without permission is not authorized.



Chapter 130 - Identity, Equality & Object Semantics

Section D — Object Semantics

ENGLISH	FRANÇAIS
What is Identity, Equality & Object Semantics? is tests identity; == asks objects for value equality through the data model.	Quel est le principe présenté dans ce chapitre ? is teste l'identité; == demande l'égalité de valeur via le modèle de données.
Why does it matter in engineering software? Use 'is None' for sentinel identity and define equality only when the domain has clear value semantics.	**Pourquoi est-ce important dans un logiciel d'ingénierie ?** Utilisez 'is None' pour l'identité d'une sentinelle et ne définissez l'égalité que si le domaine a une sémantique de valeur claire.
What design rule should be remembered? Use is for identity and == for domain-defined equality.	**Quelle règle de conception faut-il retenir ?** Utiliser is pour l'identité et == pour l'égalité métier.

Engineering Notes / Notes d'ingénierie

Engineering check: Use is for identity and == for domain-defined equality.	Vérification d'ingénierie : Utiliser is pour l'identité et == pour l'égalité métier.

Shared engineering example - code remains English only

```
from dataclasses import dataclass
@dataclass
class Reading:
    volts: float
a = Reading(5.0); b = Reading(5.0)
assert a == b and a is not b
```



© AHDS. Portfolio / educational material. Redistribution or reproduction without permission is not authorized.



Chapter 131 - Hashing Custom Objects

Section D — Object Semantics

ENGLISH	FRANÇAIS
What is Hashing Custom Objects? Hashing lets immutable value-like objects act as dictionary keys or set members when equality and hash remain consistent.	Quel est le principe présenté dans ce chapitre ? Le hachage permet aux objets immuables de type valeur d'être clés de dictionnaire ou membres d'ensemble lorsque égalité et hash restent cohérents.
Why does it matter in engineering software? Never base a hash on fields that can change while the object is in a set or dictionary.	**Pourquoi est-ce important dans un logiciel d'ingénierie ?** Ne basez jamais un hash sur des champs pouvant changer pendant que l'objet est dans un set ou dictionnaire.
What design rule should be remembered? Equal objects must hash equally; hash-relevant state must be stable.	**Quelle règle de conception faut-il retenir ?** Des objets égaux ont le même hash; les champs concernés restent stables.

Engineering Notes / Notes d'ingénierie

Engineering check: Equal objects must hash equally; hash-relevant state must be stable.	Vérification d'ingénierie : Des objets égaux ont le même hash; les champs concernés restent stables.

Shared engineering example - code remains English only

```
class Channel:
  def __init__(self, bus, address):
    self.bus, self.address = bus, address
  def __eq__(self, other):
    if not isinstance(other, Channel): return NotImplemented
    return (self.bus, self.address) == (other.bus, other.address)
  def __hash__(self): return hash((self.bus, self.address))
```



© AHDS. Portfolio / educational material. Redistribution or reproduction without permission is not authorized.



Chapter 132 - Copying Object Graphs & Custom Objects

Section D — Object Semantics

ENGLISH	FRANÇAIS
What is Copying Object Graphs & Custom Objects? Shallow copy duplicates the outer object while deep copy recursively copies reachable content according to copy protocol rules.	Quel est le principe présenté dans ce chapitre ? La copie superficielle duplique l'objet externe tandis que la copie profonde copie récursivement le contenu accessible selon le protocole copy.
Why does it matter in engineering software? Do not deep-copy live handles such as serial ports or locks; define explicit cloning for configuration/state models.	**Pourquoi est-ce important dans un logiciel d'ingénierie ?** Ne copiez pas profondément des handles actifs comme ports série ou verrous; définissez un clonage explicite pour configuration/état.
What design rule should be remembered? Deep-copy only owned state; sockets and locks require lifecycle control.	**Quelle règle de conception faut-il retenir ?** Ne copier en profondeur que l'état possédé; gérer sockets et verrous.

Engineering Notes / Notes d'ingénierie

Engineering check: Deep-copy only owned state; sockets and locks require lifecycle control.	Vérification d'ingénierie : Ne copier en profondeur que l'état possédé; gérer sockets et verrous.

Shared engineering example - code remains English only

```
from copy import copy, deepcopy
cfg = {"channels": [0, 1]}
shallow = copy(cfg); deep = deepcopy(cfg)
cfg["channels"].append(2)
assert deep["channels"] == [0, 1]
```



© AHDS. Portfolio / educational material. Redistribution or reproduction without permission is not authorized.



Chapter 133 - Object Lifetime & Garbage Collection Concepts

Section D — Object Semantics

ENGLISH	FRANÇAIS
What is Object Lifetime & Garbage Collection Concepts? Python manages object lifetime primarily through reference counting plus cyclic garbage collection in CPython.	Quel est le principe présenté dans ce chapitre ? Python gère la durée de vie des objets principalement par comptage de références et collecte des cycles dans CPython.
Why does it matter in engineering software? Do not rely on object destruction timing to close instruments, files or sockets; use explicit close or context managers.	**Pourquoi est-ce important dans un logiciel d'ingénierie ?** Ne comptez pas sur le moment de destruction pour fermer instruments, fichiers ou sockets; utilisez close explicite ou des context managers.
What design rule should be remembered? Release device resources explicitly via context managers, even on failure.	**Quelle règle de conception faut-il retenir ?** Libérer les ressources via des contextes, même en cas d'échec.

Engineering Notes / Notes d'ingénierie

Engineering check: Release device resources explicitly via context managers, even on failure.	Vérification d'ingénierie : Libérer les ressources via des contextes, même en cas d'échec.

Shared engineering example - code remains English only

```
from contextlib import contextmanager
@contextmanager
def instrument_session():
    session = {"open": True}
    try: yield session
    finally: session["open"] = False
with instrument_session() as dev: assert dev["open"]
```



© AHDS. Portfolio / educational material. Redistribution or reproduction without permission is not authorized.



Chapter 134 - Special Methods / Dunder Methods

Section E — Python Data Model & Special Methods

ENGLISH	FRANÇAIS
What is Special Methods / Dunder Methods? Special methods connect user-defined classes to Python syntax and built-in operations.	Quel est le principe présenté dans ce chapitre ? Les méthodes spéciales relient les classes utilisateur à la syntaxe Python et aux opérations intégrées.
Why does it matter in engineering software? Implement only protocols your type genuinely supports; consistency matters more than clever operator syntax.	**Pourquoi est-ce important dans un logiciel d'ingénierie ?** N'implémentez que les protocoles réellement pris en charge; la cohérence compte plus qu'une syntaxe d'opérateur astucieuse.
What design rule should be remembered? Special methods must obey Python protocol expectations.	**Quelle règle de conception faut-il retenir ?** Les méthodes spéciales doivent respecter les protocoles Python.

Engineering Notes / Notes d'ingénierie

Engineering check: Special methods must obey Python protocol expectations.	Vérification d'ingénierie : Les méthodes spéciales doivent respecter les protocoles Python.

Shared engineering example - code remains English only

```
class Register:
    def __init__(self, value): self.value = value
    def __int__(self): return self.value
    def __repr__(self): return f"Register({self.value})"
assert int(Register(7)) == 7
```



© AHDS. Portfolio / educational material. Redistribution or reproduction without permission is not authorized.



Chapter 135 - Object String Representation

Section E — Python Data Model & Special Methods

ENGLISH	FRANÇAIS
What is Object String Representation? __repr__ targets unambiguous developer-facing representation; __str__ targets readable user-facing text.	Quel est le principe présenté dans ce chapitre ? __repr__ vise une représentation non ambiguë pour le développeur; __str__ vise un texte lisible pour l'utilisateur.
Why does it matter in engineering software? Include identifying engineering state but never leak secrets or huge binary payloads.	**Pourquoi est-ce important dans un logiciel d'ingénierie ?** Incluez l'état d'ingénierie utile à l'identification sans exposer secrets ni gros payloads binaires.
What design rule should be remembered? repr should be diagnostic and avoid exposing secrets or large buffers.	**Quelle règle de conception faut-il retenir ?** repr doit aider au diagnostic sans exposer secrets ni gros tampons.

Engineering Notes / Notes d'ingénierie

Engineering check: repr should be diagnostic and avoid exposing secrets or large buffers.	Vérification d'ingénierie : repr doit aider au diagnostic sans exposer secrets ni gros tampons.

Shared engineering example - code remains English only

```
class DMM:
    def __init__(self, address): self.address = address
    def __repr__(self):
        return f"DMM(address={self.address!r})"
assert "COM4" in repr(DMM("COM4"))
```



© AHDS. Portfolio / educational material. Redistribution or reproduction without permission is not authorized.



Chapter 136 - Comparison Special Methods

Section E — Python Data Model & Special Methods

ENGLISH	FRANÇAIS
What is Comparison Special Methods? Comparison special methods define ==, !=, <, <=, > and >= behavior.	Quel est le principe présenté dans ce chapitre ? Les méthodes spéciales de comparaison définissent le comportement de ==, !=, <, <=, > et >=.
Why does it matter in engineering software? Only define ordering when the domain has a meaningful total or partial order.	**Pourquoi est-ce important dans un logiciel d'ingénierie ?** Ne définissez un ordre que si le domaine possède un ordre total ou partiel significatif.
What design rule should be remembered? Return NotImplemented for unsupported comparison types.	**Quelle règle de conception faut-il retenir ?** Retourner NotImplemented pour les comparaisons non prises en charge.

Engineering Notes / Notes d'ingénierie

Engineering check: Return NotImplemented for unsupported comparison types.	Vérification d'ingénierie : Retourner NotImplemented pour les comparaisons non prises en charge.

Shared engineering example - code remains English only

```
class Channel:
    def __init__(self, index): self.index = index
    def __eq__(self, other):
        if not isinstance(other, Channel): return NotImplemented
        return self.index == other.index
```



© AHDS. Portfolio / educational material. Redistribution or reproduction without permission is not authorized.



Chapter 137 - Operator Overloading

Section E — Python Data Model & Special Methods

ENGLISH	FRANÇAIS
What is Operator Overloading? Operator overloading maps operators to special methods such as __add__, __or__ or __mul__.	Quel est le principe présenté dans ce chapitre ? La surcharge d'opérateurs associe les opérateurs à des méthodes spéciales comme __add__, __or__ ou __mul__.
Why does it matter in engineering software? Use operators only when their domain meaning is obvious, for example units, vectors or register masks.	**Pourquoi est-ce important dans un logiciel d'ingénierie ?** Utilisez les opérateurs seulement lorsque leur sens métier est évident, par exemple unités, vecteurs ou masques de registre.
What design rule should be remembered? Operator overloads must remain predictable and type-safe.	**Quelle règle de conception faut-il retenir ?** Les opérateurs surchargés doivent rester prévisibles et typés.

Engineering Notes / Notes d'ingénierie

Engineering check: Operator overloads must remain predictable and type-safe.	Vérification d'ingénierie : Les opérateurs surchargés doivent rester prévisibles et typés.

Shared engineering example - code remains English only

```
class Mask:
   def __init__(self, value): self.value = value
   def __or__(self, other):
      if not isinstance(other, Mask): return NotImplemented
      return Mask(self.value | other.value)
assert (Mask(1) | Mask(2)).value == 3
```



© AHDS. Portfolio / educational material. Redistribution or reproduction without permission is not authorized.



Chapter 138 - Container Protocols

Section E — Python Data Model & Special Methods

ENGLISH	FRANÇAIS
What is Container Protocols? Container protocols let custom objects participate in len(), indexing, membership and iteration.	Quel est le principe présenté dans ce chapitre ? Les protocoles de conteneur permettent aux objets personnalisés de participer à len(), indexation, appartenance et itération.
Why does it matter in engineering software? A register bank or sample buffer can expose familiar container behavior without leaking internal storage.	**Pourquoi est-ce important dans un logiciel d'ingénierie ?** Une banque de registres ou un buffer d'échantillons peut exposer un comportement familier sans révéler le stockage interne.
What design rule should be remembered? Container operations must agree on indexing, length and iteration.	**Quelle règle de conception faut-il retenir ?** Indexation, taille et itération doivent être cohérentes.

Engineering Notes / Notes d'ingénierie

Engineering check: Container operations must agree on indexing, length and iteration.	Vérification d'ingénierie : Indexation, taille et itération doivent être cohérentes.

Shared engineering example - code remains English only

```
class Samples:
    def __init__(self, data): self._data = data
    def __len__(self): return len(self._data)
    def __getitem__(self, index): return self._data[index]
assert list(Samples([1, 2])) == [1, 2]
```



© AHDS. Portfolio / educational material. Redistribution or reproduction without permission is not authorized.



Chapter 139 - Iterator Protocol

Section E — Python Data Model & Special Methods

ENGLISH	FRANÇAIS
What is Iterator Protocol? The iterator protocol uses __iter__() and __next__() and signals completion with StopIteration.	Quel est le principe présenté dans ce chapitre ? Le protocole d'itérateur utilise __iter__() et __next__() et signale la fin avec StopIteration.
Why does it matter in engineering software? For streaming hardware, generators are often simpler unless a reusable stateful iterator object is required.	**Pourquoi est-ce important dans un logiciel d'ingénierie ?** Pour le matériel en streaming, les générateurs sont souvent plus simples sauf si un itérateur avec état réutilisable est requis.
What design rule should be remembered? Iterators must signal exhaustion with StopIteration.	**Quelle règle de conception faut-il retenir ?** Les itérateurs signalent la fin avec StopIteration.

Engineering Notes / Notes d'ingénierie

Engineering check: Iterators must signal exhaustion with StopIteration.	Vérification d'ingénierie : Les itérateurs signalent la fin avec StopIteration.

Shared engineering example - code remains English only

```
samples = iter([1.1, 1.2, 1.3])
first = next(samples)
remaining = list(samples)
assert first == 1.1 and len(remaining) == 2
```



© AHDS. Portfolio / educational material. Redistribution or reproduction without permission is not authorized.



Chapter 140 - Callable Objects — __call__()

Section E — Python Data Model & Special Methods

ENGLISH	FRANÇAIS
What is Callable Objects — __call__()? __call__ makes an instance callable while retaining configuration and state between calls.	Quel est le principe présenté dans ce chapitre ? __call__ rend une instance appelable tout en conservant configuration et état entre les appels.
Why does it matter in engineering software? Useful for calibrated converters, filters and command objects.	**Pourquoi est-ce important dans un logiciel d'ingénierie ?** Utile pour convertisseurs étalonnés, filtres et objets commande.
What design rule should be remembered? Callable objects are valuable when processing owns explicit state.	**Quelle règle de conception faut-il retenir ?** Les objets appelables conviennent aux traitements avec état explicite.

Engineering Notes / Notes d'ingénierie

Engineering check: Callable objects are valuable when processing owns explicit state.	Vérification d'ingénierie : Les objets appelables conviennent aux traitements avec état explicite.

Shared engineering example - code remains English only

```
class Scale:
  def __init__(self, gain): self.gain=gain
  def __call__(self, raw): return raw*self.gain
```



© AHDS. Portfolio / educational material. Redistribution or reproduction without permission is not authorized.



Chapter 141 - Context Manager Protocol

Section E — Python Data Model & Special Methods

ENGLISH	FRANÇAIS
What is Context Manager Protocol? Context managers define deterministic enter/exit behavior around a managed resource.	Quel est le principe présenté dans ce chapitre ? Les context managers définissent un comportement déterministe d'entrée/sortie autour d'une ressource gérée.
Why does it matter in engineering software? They are ideal for instrument sessions, temporary modes, locks and transaction-like hardware operations.	**Pourquoi est-ce important dans un logiciel d'ingénierie ?** Ils sont idéaux pour sessions d'instrument, modes temporaires, verrous et opérations matérielles transactionnelles.
What design rule should be remembered? Use try/finally semantics for deterministic instrument cleanup.	**Quelle règle de conception faut-il retenir ?** Utiliser try/finally pour libérer les instruments de façon déterministe.

Engineering Notes / Notes d'ingénierie

Engineering check: Use try/finally semantics for deterministic instrument cleanup.	Vérification d'ingénierie : Utiliser try/finally pour libérer les instruments de façon déterministe.

Shared engineering example - code remains English only

```
from contextlib import contextmanager
@contextmanager
def open_session():
   try: yield {"samples": [1.0, 2.0]}
   finally: pass  # release hardware here
with open_session() as s:
   trace = s["samples"]
```



© AHDS. Portfolio / educational material. Redistribution or reproduction without permission is not authorized.



Chapter 142 - Customizing Attribute Access

Section E — Python Data Model & Special Methods

ENGLISH	FRANÇAIS
What is Customizing Attribute Access? __getattribute__, __getattr__, __setattr__ and related hooks can customize attribute access.	Quel est le principe présenté dans ce chapitre ? __getattribute__, __getattr__, __setattr__ et hooks associés peuvent personnaliser l'accès aux attributs.
Why does it matter in engineering software? These hooks are powerful and easy to recurse accidentally; prefer properties/descriptors for local behavior.	**Pourquoi est-ce important dans un logiciel d'ingénierie ?** Ces hooks sont puissants et peuvent facilement récursiver; préférez propriétés/descripteurs pour un comportement local.
What design rule should be remembered? Avoid surprising side effects in custom attribute lookup.	**Quelle règle de conception faut-il retenir ?** Éviter les effets de bord inattendus lors de l'accès aux attributs.

Engineering Notes / Notes d'ingénierie

Engineering check: Avoid surprising side effects in custom attribute lookup.	Vérification d'ingénierie : Éviter les effets de bord inattendus lors de l'accès aux attributs.

Shared engineering example - code remains English only

```
class Registers:
    def read_register(self, name): return {"gain": 10}[name]
    def __getattr__(self, name):
        if name.startswith("reg_"):
            return self.read_register(name[4:])
        raise AttributeError(name)
assert Registers().reg_gain == 10
```



© AHDS. Portfolio / educational material. Redistribution or reproduction without permission is not authorized.



Chapter 143 - Descriptors

Section E — Python Data Model & Special Methods

ENGLISH	FRANÇAIS
What is Descriptors? A descriptor controls attribute access through __get__, __set__ or __delete__ at the class level.	**Qu'est-ce que Descriptors ?** Un descripteur contrôle l'accès aux attributs via __get__, __set__ ou __delete__ au niveau de la classe.
Why does it matter in engineering software? Descriptors can centralize register-field validation or unit conversion, but simple properties are easier when reuse is unnecessary.	**Pourquoi est-ce important dans un logiciel d'ingénierie ?** Les descripteurs peuvent centraliser validation de champs de registre ou conversion d'unités, mais les propriétés sont plus simples sans besoin de réutilisation.
What design rule should be remembered? Descriptors should validate shared field semantics across instances.	**Quelle règle de conception faut-il retenir ?** Les descripteurs valident les règles communes aux instances.

Engineering Notes / Notes d'ingénierie

Engineering check: Descriptors should validate shared field semantics across instances.	Vérification d'ingénierie : Les descripteurs valident les règles communes aux instances.

Shared engineering example - code remains English only

```
class VoltageField:
    def __get__(self, obj, owner):
        if obj is None: return self
        return obj._v
    def __set__(self, obj, v):
        if not 0 <= v <= 30: raise ValueError(v)
        obj._v = float(v)
```



© AHDS. Portfolio / educational material. Redistribution or reproduction without permission is not authorized.



Chapter 144 - __slots__

Section E — Python Data Model & Special Methods

ENGLISH	FRANÇAIS
What is __slots__? __slots__ declares a fixed set of instance attribute slots and can reduce per-instance memory overhead.	**Qu'est-ce que __slots__ ?** __slots__ déclare un ensemble fixe d'attributs d'instance et peut réduire le coût mémoire par instance.
Why does it matter in engineering software? Use it for measured reasons, not as a default optimization; inheritance and weak references require care.	**Pourquoi est-ce important dans un logiciel d'ingénierie ?** Utilisez-le pour des raisons mesurées, pas comme optimisation par défaut; héritage et références faibles demandent de l'attention.
What design rule should be remembered? Slots restrict ordinary attributes, not all mutation or memory allocations.	**Quelle règle de conception faut-il retenir ?** Slots limite les attributs, pas toutes les mutations ou allocations.

Engineering Notes / Notes d'ingénierie

Engineering check: Slots restrict ordinary attributes, not all mutation or memory allocations.	Vérification d'ingénierie : Slots limite les attributs, pas toutes les mutations ou allocations.

Shared engineering example - code remains English only

```
class Sample:
   __slots__ = ("t", "value")
   def __init__(self, t, value):
      self.t, self.value = t, value
s = Sample(0.0, 3.3)
```



© AHDS. Portfolio / educational material. Redistribution or reproduction without permission is not authorized.



Chapter 145 - Object Creation — __new__()

Section E — Python Data Model & Special Methods

ENGLISH	FRANÇAIS
What is Object Creation — __new__()? __new__ creates and returns the instance before __init__ initializes it.	Quel est le principe présenté dans ce chapitre ? __new__ crée et retourne l'instance avant que __init__ ne l'initialise.
Why does it matter in engineering software? Override it rarely: immutable subclasses, controlled construction or metaprogramming are typical cases.	**Pourquoi est-ce important dans un logiciel d'ingénierie ?** Surchargez-le rarement: sous-classes immuables, construction contrôlée ou métaprogrammation sont des cas typiques.
What design rule should be remembered? Use __new__ only when instance creation itself must be customized.	**Quelle règle de conception faut-il retenir ?** Réserver __new__ aux besoins de personnalisation de la création.

Engineering Notes / Notes d'ingénierie

Engineering check: Use __new__ only when instance creation itself must be customized.	Vérification d'ingénierie : Réserver __new__ aux besoins de personnalisation de la création.

Shared engineering example - code remains English only

```
class Channel(int):
  def __new__(cls, n):
    if not 0 <= n < 16: raise ValueError(n)
    return super().__new__(cls, n)
```



© AHDS. Portfolio / educational material. Redistribution or reproduction without permission is not authorized.



Chapter 146 - Metaclasses

Section E — Python Data Model & Special Methods

ENGLISH	FRANÇAIS
What is Metaclasses? A metaclass controls class creation; type is Python's default metaclass.	**Qu’est-ce que Metaclasses ?** Une métaclasse contrôle la création des classes; type est la métaclasse par défaut de Python.
Why does it matter in engineering software? Metaclasses are advanced infrastructure. Prefer decorators or __init_subclass__ unless class-creation control is genuinely required.	**Pourquoi est-ce important dans un logiciel d’ingénierie ?** Les métaclasses sont une infrastructure avancée. Préférez décorateurs ou __init_subclass__ sauf besoin réel de contrôler la création de classes.
What design rule should be remembered? Avoid metaclasses when simpler registries or class decorators suffice.	**Quelle règle de conception faut-il retenir ?** Éviter les métaclasses si un registre ou décorateur suffit.

Engineering Notes / Notes d’ingénierie

Engineering check: Avoid metaclasses when simpler registries or class decorators suffice.	Vérification d’ingénierie : Éviter les métaclasses si un registre ou décorateur suffit.

Shared engineering example - code remains English only

```
class Registry(type):
  def __new__(mcls, name, bases, ns):
    return super().__new__(mcls, name, bases, ns)
```



© AHDS. Portfolio / educational material. Redistribution or reproduction without permission is not authorized.



Chapter 147 - Class Decorators & Dynamic Class Construction

Section E — Python Data Model & Special Methods

ENGLISH	FRANÇAIS
What is Class Decorators & Dynamic Class Construction? Class decorators transform or register a class after its creation; dynamic construction can build classes from runtime metadata.	Quel est le principe présenté dans ce chapitre ? Les décorateurs de classe transforment ou enregistrent une classe après sa création; la construction dynamique peut créer des classes depuis des métadonnées d'exécution.
Why does it matter in engineering software? Useful for driver registries, but keep registration deterministic and discoverable.	**Pourquoi est-ce important dans un logiciel d'ingénierie ?** Utile pour registres de pilotes, mais gardez l'enregistrement déterministe et découvrable.
What design rule should be remembered? Registries must reject duplicate model identifiers explicitly.	**Quelle règle de conception faut-il retenir ?** Les registres doivent refuser les identifiants de modèle dupliqués.

Engineering Notes / Notes d'ingénierie

Engineering check: Registries must reject duplicate model identifiers explicitly.	Vérification d'ingénierie : Les registres doivent refuser les identifiants de modèle dupliqués.

Shared engineering example - code remains English only

```
DRIVERS = {}
def register(cls):
    DRIVERS[cls.model] = cls
    return cls
@register
class SimulatedDriver:
    model = "SIM"
```



© AHDS. Portfolio / educational material. Redistribution or reproduction without permission is not authorized.



Chapter 148 - OOP for Hardware, Drivers & Instruments

Section F — Engineering OOP

ENGLISH	FRANÇAIS
What is OOP for Hardware, Drivers & Instruments? Engineering OOP separates transport, protocol, device behavior, configuration and measurements into testable responsibilities.	**Qu'est-ce que OOP for Hardware, Drivers & Instruments ?** La POO d'ingénierie sépare transport, protocole, comportement de l'appareil, configuration et mesures en responsabilités testables.
Why does it matter in engineering software? Inject transports so the same driver can run against serial hardware, TCP, loopback or a simulator.	**Pourquoi est-ce important dans un logiciel d'ingénierie ?** Injectez les transports afin que le même pilote fonctionne avec matériel série, TCP, loopback ou simulateur.
What design rule should be remembered? Drivers must validate output limits and separate transport errors.	**Quelle règle de conception faut-il retenir ?** Les pilotes valident les limites de sortie et isolent les erreurs de transport.

Engineering Notes / Notes d'ingénierie

Engineering check: Drivers must validate output limits and separate transport errors.	Vérification d'ingénierie : Les pilotes valident les limites de sortie et isolent les erreurs de transport.

Shared engineering example - code remains English only

```
class PowerSupply:
  def __init__(self, transport): self.io = transport
  def set_voltage(self, v):
    if not 0 <= v <= 30: raise ValueError(v)
    self.io.write(f"VOLT {v}")
```



© AHDS. Portfolio / educational material. Redistribution or reproduction without permission is not authorized.



Chapter 149 - Complete Object-Oriented Engineering Architecture

Section F — Engineering OOP

ENGLISH	FRANÇAIS
What is Complete Object-Oriented Engineering Architecture? A complete architecture defines stable boundaries from application services down to device transport and back up through typed results.	Quel est le principe présenté dans ce chapitre ? Une architecture complète définit des frontières stables depuis les services applicatifs jusqu'au transport matériel et remonte via des résultats typés.
Why does it matter in engineering software? Design for simulation, logging, fault isolation, controlled shutdown and replaceable device implementations from the beginning.	**Pourquoi est-ce important dans un logiciel d'ingénierie ?** Concevez dès le départ pour simulation, journalisation, isolation des pannes, arrêt contrôlé et implémentations d'appareils remplaçables.
What design rule should be remembered? Inject dependencies to test the complete control path without hardware.	**Quelle règle de conception faut-il retenir ?** Injecter les dépendances pour tester la chaîne sans matériel.

Engineering Notes / Notes d'ingénierie

Engineering check: Inject dependencies to test the complete control path without hardware.	Vérification d'ingénierie : Injecter les dépendances pour tester la chaîne sans matériel.

Shared engineering example - code remains English only

```
class FakeTransport:
    def __init__(self): self.commands = []
    def write(self, command): self.commands.append(command)
class PowerSupply:
    def __init__(self, io): self.io = io
    def set_voltage(self, volts): self.io.write(f"VOLT {volts}")
psu = PowerSupply(FakeTransport()); psu.set_voltage(5.0)
```



© AHDS. Portfolio / educational material. Redistribution or reproduction without permission is not authorized.